Utica University | Department of Computer Science | Fall 2026
CSC 618: Information Retrieval | Week 10 Assignment

Submission 4: Part C, Experiment on a Real Online IR Dataset

Shaurav Parajuli | November 2026

1. Summary

I ran TF-IDF, BM25 and a Dirichlet Language Model on the Cranfield test collection using Apache Lucene. With default parameters, BM25 (MAP 0.2952) and TF-IDF (MAP 0.2931) were statistically indistinguishable, and both were significantly better than LM-Dirichlet (MAP 0.2341). All values are from actual runs; nothing is estimated.

2. Dataset description and source

Table 1: Cranfield collection.

Documents	1400 aeronautical engineering abstracts; 143285 tokens after analysis, 5172 unique terms, mean 102.3 terms per document
Queries	225 natural-language questions (mean 18.0 words)
Qrels	1837 rows; 1612 positive; 7.16 relevant per query on average (range 1–39)
Why chosen	Public, has all four required components, and is a historic benchmark [5]. TREC document sets usually require a license [6]
Source used	TREC-format copy: https://github.com/oussbenk/cranfield-trec-dataset (commit 1208e6e). Original: http://ir.dcs.gla.ac.uk/resources/test_collections/cran/

Data validation. The original site could not be reached from my working environment, so I used the TRECformat copy and checked it. Its document, query and qrel counts match the ir_datasets metadata for Cranfield [7] (1400, 225, 1837), and its 225 rows labelled 0 match that metadata's 225 "no interest" (−1) rows. This copy has flattened the other grades to 1, apart from one row labelled 3, which I treated as relevant. Two documents (471, 995) have no abstract, so their titles were indexed.

3. Experimental setup and methodology

Table 2: Setup, kept the same for all models.

Engine	Apache Lucene 10.5.0, Java 21 (jar from Anserini 2.3.0 [8], distributed with Pyserini 2.4.0)
Analysis	EnglishAnalyzer on documents and queries (default stop words, Porter stemmer)
Indexed text	Abstract field, which already begins with the title in the source
Query	Topic text analysed, then all terms OR-ed (BooleanQuery SHOULD)
Depth	Top 1000; mean 891 returned, since only documents with a query term match
Saved results	TREC-format run files (results/partc_*.run)
Evaluation	Part B functions; set metrics on the full list and at rank 10; AP over the list; nDCG with binary gain
Testing	Paired two-sided randomization test (100,000 sign flips, seed 618) and Wilcoxon signed-rank [9]

Table 3: Models. *t* is a query term, *f* its frequency in document *d*, |*d*| the document length.

Model	Scoring idea	Parameters
TF-IDF (ClassicSimilarity)	$\sqrt{f} \cdot \mathrm{idf}^2 \cdot |d|^{-1/2}$, summed over terms	defaults
BM25 (BM25Similarity) [10]	idf times saturating f, normalised by $|d|$ relative to the mean	k_1=1.2, b=0.75
LM-Dirichlet (LMDirichletSimilarity) [11]	Term probability in d smoothed by the collection model	μ=2000

4. Results

Table 4: Mean metrics over 225 queries (best in bold, ties not marked).

Metric	TF-IDF	BM25	LM-Dirichlet
MAP	0.2931	0.2952	0.2341
MRR	0.5327	0.5204	0.4557
P@3	0.3452	0.3570	0.2726
P@10	0.2244	0.2284	0.1840
Recall@10	0.3806	0.3909	0.3248
F1@10	0.2544	0.2600	0.2119
nDCG@5	0.3663	0.3651	0.2858
nDCG@10	0.3690	0.3741	0.3041
Precision (full list)	0.0081	0.0081	0.0081
Recall (full list)	0.9513	0.9523	0.9515
F1 (full list)	0.0160	0.0160	0.0160

Table 5: Paired tests over 225 queries. W/L/T counts queries where the first model scored higher, lower or equal.

Comparison	Metric	Mean diff.	W/L/T	Randomization p	Wilcoxon p
BM25 vs TF-IDF	AP	+0.0021	113/101/11	0.7492	0.7537
BM25 vs TF-IDF	nDCG@10	+0.0050	92/79/54	0.5228	0.4959
BM25 vs TF-IDF	RR	-0.0123	62/64/99	0.4792	0.7383
BM25 vs TF-IDF	P@10	+0.0040	40/29/156	0.4446	0.4442
BM25 vs LM-Dirichlet	AP	+0.0611	169/49/7	$< 10^{-5}$	$< 10^{-5}$
BM25 vs LM-Dirichlet	nDCG@10	+0.0700	133/50/42	$< 10^{-5}$	$< 10^{-5}$
BM25 vs LM-Dirichlet	RR	+0.0647	96/41/88	0.00056	0.00009
BM25 vs LM-Dirichlet	P@10	+0.0444	87/22/116	$< 10^{-5}$	$< 10^{-5}$
TF-IDF vs LM-Dirichlet	AP	+0.0589	157/62/6	$< 10^{-5}$	$< 10^{-5}$
TF-IDF vs LM-Dirichlet	nDCG@10	+0.0649	123/68/34	$< 10^{-5}$	$< 10^{-5}$
TF-IDF vs LM-Dirichlet	RR	+0.0770	94/56/75	0.0017	0.0023
TF-IDF vs LM-Dirichlet	P@10	+0.0404	90/31/104	$< 10^{-5}$	$< 10^{-5}$

With a Bonferroni threshold of $0.05/12 = 0.0042$, all eight comparisons involving LM-Dirichlet remain significant and none of the BM25 vs TF-IDF comparisons does.

Table 6: Query-level behaviour.

	TF-IDF	BM25	LM-Dirichlet

Queries where the model alone has the best AP	99	75	41
Queries with no relevant document in the top 10	34	32	50

The three models tied for best AP on 10 queries. Mean top-10 overlap: BM25–TF-IDF 0.762, BM25–LM 0.596, TF-IDF–LM 0.490.

Table 7: Five queries with the largest AP spread across models.

ID	Query	Rel.	TF-IDF	BM25	LM
52	what is the available information pertaining to the effect of slight rarefaction on boundary layer flows (the ?slip? effect) .	4	0.812	0.533	0.118
165	are the stable profiles of a compressible boundary layer induced by a moving wave known .	2	0.833	0.583	0.161
81	what are wind-tunnel corrections for a two-dimensional aerofoil mounted off-centre in a tunnel .	2	0.417	0.833	0.212
143	what general solutions for the stresses in pressurized shells of revolution are available .	2	0.417	0.625	0.111
119	what is the effect of initial axisymmetric deviations from circularity on the non linear (large-deflection) load-deflection response of cylinders under hydrostatic pressure .	1	1.000	1.000	0.500

Table 8: Parameter sensitivity. Exploratory: chosen after viewing test queries, with no separate tuning set.

Configuration	MAP	MRR	nDCG@10
BM25, b=0.25	0.2728	0.4993	0.3492
BM25, b=0.5	0.2850	0.5100	0.3609
BM25, b=0.75 (default)	0.2952	0.5204	0.3741
BM25, b=1.0	0.2938	0.5224	0.3716
LM-Dirichlet, μ=50	0.2757	0.5173	0.3548
LM-Dirichlet, μ=100	0.2724	0.5083	0.3489
LM-Dirichlet, μ=200	0.2666	0.4908	0.3410
LM-Dirichlet, μ=500	0.2545	0.4835	0.3304
LM-Dirichlet, μ=1000	0.2449	0.4709	0.3176
LM-Dirichlet, μ=2000 (default)	0.2341	0.4557	0.3041
LM-JM, λ=0.1	0.2591	0.5079	0.3372
LM-JM, λ=0.3	0.2719	0.5113	0.3534
LM-JM, λ=0.7	0.2758	0.5074	0.3527

5. Why performance differs

- BM25 and TF-IDF behave alike. Their top-10 lists overlap by about three quarters. Both use IDF and length normalisation, and Cranfield abstracts are short with mostly low term frequencies, so BM25's term-frequency saturation has few chances to matter. This explanation is an inference from the data; I did not test it directly.
- LM-Dirichlet is the outlier. MAP improves steadily as μ falls from 2000 to 50 (0.2341 to 0.2757), which fits over-smoothing, as μ=2000 is about 20 times the mean document length. Tuning does not close the gap to BM25, and Jelinek-Mercer smoothing peaks at a similar value (0.2758). Lucene's model, which I confirmed by inspecting the class bytecode, clamps negative scores to zero and scores matched terms only; this may also limit performance, but I did not isolate its effect.

- Means hide query-level differences. TF-IDF is best on 99 queries and BM25 on 75, and on query 81 BM25's AP is double TF-IDF's while on query 52 TF-IDF leads BM25 by 0.28.

6. Verification

My metrics matched the independent ranx library [12] to six decimal places on all eight metrics compared (three shown below). A hand check also agrees: for query 81 under BM25, relevant documents 799 and 672 are at ranks 1 and 3, so AP =(1/1+2/3)/2 = 0.8333 and RR = 1, matching the program.

Table 9: Own implementation vs ranx.

Model	MAP (own)	MAP (ranx)	MRR (own)	MRR (ranx)	nDCG@10 (own)	nDCG@10 (ranx)
TF-IDF	0.293067	0.293067	0.532660	0.532660	0.369045	0.369045
BM25	0.295186	0.295186	0.520377	0.520377	0.374072	0.374072
LM-Dirichlet	0.234132	0.234132	0.455700	0.455700	0.304098	0.304098

7. Limitations

- The collection is small, old and domain-specific, so these rankings need not hold for other search tasks.
- The qrel copy is binary, so nDCG here does not measure graded relevance. The original graded file would be needed for that.
- Unjudged documents count as non-relevant, and the Cranfield judgments are not the product of modern pooling.
- The main comparison uses default parameters; the sensitivity table was not produced with a separate tuning set.
- Lucene's versions differ from textbook forms, and the analyzer leaves question words such as what in queries.
- No user study was involved, and speed and index size were not compared.

References

[1] C. D. Manning et al., *Introduction to Information Retrieval*, Cambridge Univ. Press, 2008, Ch. 8.

[2] Apache Lucene, https://lucene.apache.org/.

[3] Cranfield, TREC-format copy, https://github.com/oussbenk/cranfield-trec-dataset.

[4] Cranfield, original files, http://ir.dcs.gla.ac.uk/resources/test_collections/cran/.

[5] C. Cleverdon, "The Cranfield tests on index language devices," *Aslib Proc.*, 19(6), 1967.

[6] NIST TREC FAQ, https://trec.nist.gov/faq.html.

[7] S. MacAvaney et al., "Simplified data wrangling with ir_datasets," SIGIR 2021.

[8] J. Lin et al., "Pyserini: A Python toolkit for reproducible information retrieval research with sparse and dense representations," SIGIR 2021.

[9] M. Smucker, J. Allan, B. Carterette, "A comparison of statistical significance tests for information retrieval evaluation," CIKM 2007.

[10] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Found. Trends Inf. Retr.*, 3(4), 2009.

[11] C. Zhai and J. Lafferty, "A study of smoothing methods for language models applied to information retrieval," *ACM TOIS*, 22(2), 2004.

[12] E. Bassani, "ranx: A blazing-fast Python library for ranking evaluation and comparison," ECIR 2022.

Appendix A: Code

Complete project with run files: csc618-week10-ir-evaluation.zip.

prepare_cranfield.py

```
"""Convert the TREC-XML copy of Cranfield to flat files and validate it.

Query IDs are renumbered 1..225 by file position: the raw <num> values are non-consecutive (1, 2, 4, 8, ...) while the qrels number
queries consecutively.

Usage: python prepare_cranfield.py <cranfield_repo_dir> <output_dir>
""" import collections
import json import re
import sys


def clean(text):
    return re.sub(r"\s+", " ", text).strip()


def field(record, tag):
    match = re.search(fr"<{tag}>(.*?)</{tag}>", record, flags=re.S) return clean(match[1]) if match else ""


def read_docs(path):
    raw = open(path, encoding="utf-8").read() records = re.findall(r"<doc>(.*?)</doc>", raw, flags=re.S) return [(field(r,
    "docno"), field(r, "title"), field(r, "text")) for r in records]


def read_queries(path):
    raw = open(path, encoding="utf-8").read() pattern = r"<num>\s*(\d+)\s*</num>\s*<title>(.*?)</title>" return [(int(number),
    clean(text)) for number, text in re.findall(pattern, raw, flags=re.S)]


def main(root, out):
    docs = read_docs(f"{root}/cran.all.1400.xml") queries = read_queries(f"{root}/cran.qry.xml") qrels =
    [line.split() for line in open(f"{root}/cranqrel.trec.txt")]

    with open(f"{out}/docs.tsv", "w") as f:
        for doc_id, title, text in docs:
            f.write(f"{doc_id}\t{title}\t{text}\n") with
    open(f"{out}/queries.tsv", "w") as f:
        for sequential_id, (number, text) in enumerate(queries, 1):
            f.write(f"{sequential_id}\t{number}\t{text}\n") with
    open(f"{out}/qrels.txt", "w") as f:
        for query, _, doc, relevance in qrels:
            f.write(f"{query} 0 {doc} {relevance}\n")

    doc_ids = {d[0] for d in docs} relevant = collections.Counter(q for q, _, d, r in qrels if int(r) > 0) stats = {
        "docs": len(docs),
        "unique_doc_ids": len(doc_ids),
        "empty_text_docs": [d[0] for d in docs if not d[2]],
        "title_is_prefix_of_text": sum(d[2].startswith(d[1][:40]) for d in docs),
        "queries": len(queries),
        "raw_max_query_num": max(number for number, _ in queries),
        "qrel_rows": len(qrels),
        "qrel_query_ids": len({r[0] for r in qrels}),
        "qrel_max_query_id": max(int(r[0]) for r in qrels),
        "qrel_label_counts": dict(collections.Counter(r[3] for r in qrels)),
        "qrel_docs_missing": sum(r[2] not in doc_ids for r in qrels),
        "queries_with_relevant": len(relevant),
```

```
            "relevant_per_query_mean": sum(relevant.values()) / max(1, len(relevant)),
            "relevant_per_query_min": min(relevant.values()),
            "relevant_per_query_max": max(relevant.values()),
        } json.dump(stats, open(f"{out}/dataset_stats.json", "w"), indent=1) print(json.dumps(stats,
        indent=1))


if __name__ == "__main__":
        main(sys.argv[1], sys.argv[2])
```

LuceneExperiment.java

```
import java.io.IOException; import java.io.PrintWriter; import java.nio.file.Files; import java.nio.file.Path; import
java.nio.file.Paths; import java.util.ArrayList; import java.util.Collections; import java.util.HashSet; import
java.util.LinkedHashMap; import java.util.List; import java.util.Locale; import java.util.Map; import java.util.Set;
import org.apache.lucene.analysis.Analyzer; import org.apache.lucene.analysis.TokenStream; import
org.apache.lucene.analysis.en.EnglishAnalyzer; import
org.apache.lucene.analysis.tokenattributes.CharTermAttribute; import
org.apache.lucene.analysis.tokenattributes.PositionIncrementAttribute; import
org.apache.lucene.document.Document; import org.apache.lucene.document.Field; import
org.apache.lucene.document.IntPoint; import org.apache.lucene.document.StringField; import
org.apache.lucene.document.TextField; import org.apache.lucene.index.DirectoryReader; import
org.apache.lucene.index.IndexReader; import org.apache.lucene.index.IndexWriter; import
org.apache.lucene.index.IndexWriterConfig; import org.apache.lucene.index.MultiTerms; import
org.apache.lucene.index.StoredFields; import org.apache.lucene.index.Term; import
org.apache.lucene.search.BooleanClause; import org.apache.lucene.search.BooleanQuery; import
org.apache.lucene.search.FuzzyQuery; import org.apache.lucene.search.IndexSearcher; import
org.apache.lucene.search.MatchAllDocsQuery; import org.apache.lucene.search.MatchNoDocsQuery; import
org.apache.lucene.search.PhraseQuery; import org.apache.lucene.search.PrefixQuery; import
org.apache.lucene.search.Query; import org.apache.lucene.search.ScoreDoc; import
org.apache.lucene.search.TermQuery; import org.apache.lucene.search.TermRangeQuery; import
org.apache.lucene.search.WildcardQuery; import org.apache.lucene.search.similarities.BM25Similarity; import
org.apache.lucene.search.similarities.ClassicSimilarity; import
org.apache.lucene.search.similarities.LMDirichletSimilarity; import
org.apache.lucene.search.similarities.LMJelinekMercerSimilarity; import
org.apache.lucene.search.similarities.Similarity; import org.apache.lucene.store.ByteBuffersDirectory; import
org.apache.lucene.store.Directory;

/**
 *      Indexes Cranfield in Lucene, then produces all ranked result files.
 *
 *      <p>Self Study 1: eight query classes per topic (top 100). Part C: retrieval models (top 1000). *
```

```
* <p>Usage: java LuceneExperiment &lt;dataDir&gt; &lt;resultsDir&gt;
 */ public class LuceneExperiment {
 private static final String FIELD = "contents"; private static final String LENGTH_FIELD =
 "len"; private static final Analyzer ANALYZER = new EnglishAnalyzer();

 /** One analysed token and its position (positions keep gaps left by removed stop words). */ record Token(String term, int position) {}

 /** Document-length summary used by the numeric range query. */ record LengthStats(double mean,
 int q1, int q3) {}

 public static void main(String[] args) throws Exception {
     Path dataDir = Paths.get(args[0]);
     Path resultsDir = Paths.get(args[1]); Files.createDirectories(resultsDir);

     Directory directory = new ByteBuffersDirectory();
     LengthStats lengths = buildIndex(dataDir, resultsDir, directory); List<String[]> queries = readQueries(dataDir);

     try (IndexReader reader = DirectoryReader.open(directory)) {
         writeIndexStats(reader, lengths, resultsDir); IndexSearcher bm25Searcher = new
         IndexSearcher(reader); bm25Searcher.setSimilarity(new BM25Similarity());

         runQueryTypeExperiment(reader, bm25Searcher, queries, lengths, resultsDir); runDemoQueries(reader,
     bm25Searcher, lengths, resultsDir); runRetrievalModels(reader, queries, resultsDir); }
 }

 // ---------------------------------------------------------------- Indexing

 /** Indexes every document and also saves its analysed terms for the KLD/JSD calculations. */ private static LengthStats buildIndex(Path
 dataDir, Path resultsDir, Directory directory) throws IOException {
     List<Integer> lengths = new ArrayList<>();
     IndexWriterConfig config = new IndexWriterConfig(ANALYZER).setSimilarity(new BM25Similarity());

     try (IndexWriter writer = new IndexWriter(directory, config);
             PrintWriter analysed = new PrintWriter(resultsDir.resolve("docs_analyzed.tsv").toFile())) { for (String line :
         Files.readAllLines(dataDir.resolve("docs.tsv"))) {
             String[] parts = line.split("\t", -1);
             String id = parts[0];
             String title = parts[1];
             String text = parts[2];
             // The <text> field already begins with the title; two documents have no text at all.
             String body = text.isEmpty() ? title : text;

             List<Token> tokens = analyze(body); lengths.add(tokens.size());

             Document document = new Document(); document.add(new StringField("id", id,
             Field.Store.YES)); document.add(new TextField(FIELD, body, Field.Store.NO));
             document.add(new IntPoint(LENGTH_FIELD, tokens.size()));
             writer.addDocument(document);
```

```
            StringBuilder row = new StringBuilder(id).append('\t'); for (Token token : tokens) {
                row.append(token.term()).append(' ');
            }
            analysed.println(row.toString().trim());
        }
    }

    Collections.sort(lengths);
    int q1 = lengths.get(lengths.size() / 4); int q3 = lengths.get(3 * lengths.size() / 4); double mean =
    lengths.stream().mapToInt(Integer::intValue).average().orElse(0); return new LengthStats(mean, q1, q3);
}

private static void writeIndexStats(IndexReader reader, LengthStats lengths, Path resultsDir) throws IOException {
    try (PrintWriter out = new PrintWriter(resultsDir.resolve("index_stats.txt").toFile())) {
        out.printf(
                Locale.ROOT,
                "lucene=%s docs=%d vocab=%d tokens=%d meanLen=%.2f q1=%d q3=%d%n",
                org.apache.lucene.util.Version.LATEST, reader.numDocs(),
                MultiTerms.getTerms(reader, FIELD).size(),
                reader.getSumTotalTermFreq(FIELD), lengths.mean(),
                lengths.q1(), lengths.q3());
    }
}

// ---------------------------------------------------------------- Helpers

/** Applies the same analysis to queries as to documents (stop words, Porter stemming). */ private static List<Token> analyze(String
text) throws IOException { List<Token> tokens = new ArrayList<>();
    try (TokenStream stream = ANALYZER.tokenStream(FIELD, text)) {
        CharTermAttribute term = stream.addAttribute(CharTermAttribute.class);
        PositionIncrementAttribute increment = stream.addAttribute(PositionIncrementAttribute.class); stream.reset(); int position = -1; while
        (stream.incrementToken()) {
            position += increment.getPositionIncrement(); tokens.add(new Token(term.toString(),
            position));
        } stream.end();
    } return tokens;
}

private static List<String[]> readQueries(Path dataDir) throws IOException { List<String[]> queries = new
    ArrayList<>(); for (String line : Files.readAllLines(dataDir.resolve("queries.tsv"))) {
        queries.add(line.split("\t", -1)); // sequential id, original number, text
    } return queries;
}

private static Term term(String text) {
    return new Term(FIELD, text);
}
```

```
/** OR of all query terms: a document needs at least one of them to match. */ private static Query
orQuery(List<Token> tokens) { BooleanQuery.Builder builder = new BooleanQuery.Builder(); for (Token token :
tokens) {
        builder.add(new TermQuery(term(token.term())), BooleanClause.Occur.SHOULD);
    } return builder.build();
}

/** Inverse document frequency, or -1 when the term is not in the index. */ private static double idf(IndexReader reader,
String text) throws IOException {
    int documentFrequency = reader.docFreq(term(text));
    return documentFrequency == 0 ? -1 : Math.log((reader.numDocs() + 1.0) / documentFrequency);
}

/** Runs a query and writes results in TREC format: qid Q0 docid rank score tag. */
private static void writeRun(
        IndexSearcher searcher, Query query, int queryId, String tag, int depth, PrintWriter out) throws IOException {
    StoredFields fields = searcher.getIndexReader().storedFields(); ScoreDoc[] hits =
    searcher.search(query, depth).scoreDocs; for (int i = 0; i < hits.length; i++) {
        String docId = fields.document(hits[i].doc).get("id"); out.printf(Locale.ROOT, "%d Q0 %s %d %.6f %s%n", queryId, docId, i + 1,
        hits[i].score, tag);
    }
}

// ---------------------------------------------------------------------- Self Study 1

private static String rarestTerm(IndexReader reader, List<Token> tokens) throws IOException { String best = null; double bestIdf = -1; for
    (Token token : tokens) {
        double value = idf(reader, token.term()); if (value > bestIdf) {
        bestIdf = value; best = token.term();
        } } return best;
}

/** Adjacent bigram with the highest IDF sum that matches at least one document. */ private static Query bestPhrase(IndexReader reader,
IndexSearcher searcher, List<Token> tokens) throws IOException {
    PhraseQuery best = null; double bestValue
    = -1e9;
    for (int i = 0; i + 1 < tokens.size(); i++) {
        Token first = tokens.get(i); Token second = tokens.get(i + 1);
        if (first.term().equals(second.term())) {
            continue;
        }
        PhraseQuery candidate = new
                PhraseQuery.Builder()
                        .add(term(first.term()), 0)
                        .add(term(second.term()), second.position() - first.position())
                        .build(); double value = idf(reader, first.term()) + idf(reader, second.term()); if
        (searcher.count(candidate) > 0 && value > bestValue) {
            bestValue = value; best =
            candidate;
```

```
        } }
    return best == null ? new MatchNoDocsQuery("no matching bigram") : best;
}

/** Builds each query class for one topic. Uses topic text and corpus statistics, never qrels. */ private static Map<String, Query> buildQueries(
        IndexReader reader, IndexSearcher searcher, List<Token> tokens, LengthStats lengths) throws IOException {
    String rarest = rarestTerm(reader, tokens);
    String prefix = rarest.substring(0, Math.min(4, rarest.length()));
    String wildcard = rarest.length() >= 5
                    ? rarest.substring(0, 3) + "*" + rarest.charAt(rarest.length() - 1) : rarest + "*";

    BooleanQuery.Builder bool = new BooleanQuery.Builder(); bool.add(new
    TermQuery(term(rarest)), BooleanClause.Occur.MUST); for (Token token : tokens) {
        if (!token.term().equals(rarest)) {
            bool.add(new TermQuery(term(token.term())), BooleanClause.Occur.SHOULD);
        }
    }
    Map<String, Query> queries = new LinkedHashMap<>(); queries.put("Term", new TermQuery(term(rarest))); queries.put("Phrase",
    bestPhrase(reader, searcher, tokens)); queries.put("Range-Numeric", IntPoint.newRangeQuery(LENGTH_FIELD, lengths.q1(), lengths.q3()));
    queries.put("Range-Lexical", TermRangeQuery.newStringRange(FIELD, prefix, rarest, true, true)); queries.put("Boolean", bool.build());
    queries.put("Prefix", new PrefixQuery(term(prefix))); queries.put("Wildcard", new WildcardQuery(term(wildcard))); queries.put("Fuzzy",
    new FuzzyQuery(term(rarest), 2)); queries.put("MatchAll", new MatchAllDocsQuery()); queries.put("OR-Baseline", orQuery(tokens)); //
    reference condition, not a ninth class return queries;
}

private static void runQueryTypeExperiment(
        IndexReader reader,
        IndexSearcher searcher,
        List<String[]> topics,
        LengthStats lengths, Path
        resultsDir) throws IOException {
    String[] types = {
        "Term", "Phrase", "Range-Numeric", "Range-Lexical", "Boolean",
        "Prefix", "Wildcard", "Fuzzy", "MatchAll", "OR-Baseline" };
    Map<String, PrintWriter> runFiles = new LinkedHashMap<>(); for (String type : types) {
        runFiles.put(type, new PrintWriter(resultsDir.resolve("ss1_" + type + ".run").toFile()));
    } try (PrintWriter generated = new PrintWriter(resultsDir.resolve("ss1_generated_queries.tsv").toFile());
            PrintWriter counts = new PrintWriter(resultsDir.resolve("ss1_hit_counts.tsv").toFile())) { generated.println("qid\ttype\tquery");
        counts.println("qid\ttype\thits"); for (String[] topic : topics) {
            int id = Integer.parseInt(topic[0]); List<Token> tokens =
            analyze(topic[2]);
            for (var entry : buildQueries(reader, searcher, tokens, lengths).entrySet()) {
                String type = entry.getKey(); Query query = entry.getValue(); writeRun(searcher, query, id,
            type, 100, runFiles.get(type)); generated.println(id + "\t" + type + "\t" + query); counts.println(id
            + "\t" + type + "\t" + searcher.count(query)); }
        } } runFiles.values().forEach(PrintWriter::close);
}

/** Hand-written queries showing how each class behaves, including word order and misspelling. */ private static void runDemoQueries(
```

```
    IndexReader reader, IndexSearcher searcher, LengthStats lengths, Path resultsDir) throws IOException {
Map<String, Query> demo = new LinkedHashMap<>(); demo.put("TermQuery boundari", new
TermQuery(term("boundari")));
demo.put("PhraseQuery \"boundari layer\"", new PhraseQuery(FIELD, "boundari", "layer")); demo.put(
        "PhraseQuery \"layer boundari\" (reversed)", new PhraseQuery(FIELD, "layer", "boundari")); demo.put(
        "PhraseQuery slop=2 \"layer boundari\"", new PhraseQuery(2, FIELD, "layer", "boundari")); demo.put(
        "TermRangeQuery [bound TO boundz]",
        TermRangeQuery.newStringRange(FIELD, "bound", "boundz", true, true)); demo.put(
        "IntPoint range len in [" + lengths.q1() + "," + lengths.q3() + "]",
        IntPoint.newRangeQuery(LENGTH_FIELD, lengths.q1(), lengths.q3())); demo.put(
        "BooleanQuery +boundari +layer -transit",
```

```
            new BooleanQuery.Builder()
                    .add(new TermQuery(term("boundari")), BooleanClause.Occur.MUST)
                    .add(new TermQuery(term("layer")), BooleanClause.Occur.MUST)
                    .add(new TermQuery(term("transit")), BooleanClause.Occur.MUST_NOT)
                    .build());
        demo.put("PrefixQuery aero*", new PrefixQuery(term("aero"))); demo.put("WildcardQuery t?rbul*", new
        WildcardQuery(term("t?rbul*"))); demo.put("FuzzyQuery turbulanc~2 (misspelt)", new FuzzyQuery(term("turbulanc"), 2));
        demo.put("TermQuery turbulanc (misspelt)", new TermQuery(term("turbulanc"))); demo.put("MatchAllDocsQuery", new
        MatchAllDocsQuery());

        StoredFields fields = reader.storedFields(); try (PrintWriter out = new PrintWriter(resultsDir.resolve("ss1_demo.tsv").toFile())) {
        out.println("query\thits\tdistinctTop100Scores\ttopScore\ttop5"); for (var entry : demo.entrySet()) {
            ScoreDoc[] hits = searcher.search(entry.getValue(), 100).scoreDocs; Set<Float> distinctScores =
            new HashSet<>(); for (ScoreDoc hit : hits) {
                distinctScores.add(hit.score);
            }
            StringBuilder top = new StringBuilder(); for (int i = 0; i < Math.min(5, hits.length); i++) { top.append(i > 0 ? "," :
            "").append(fields.document(hits[i].doc).get("id"));
            } out.printf(
                    Locale.ROOT, "%s\t%d\t%d\t%.3f\t%s%n",
                    entry.getKey(), searcher.count(entry.getValue()),
                    distinctScores.size(), hits.length > 0 ? hits[0].score :
                    0, top);
        }
    }
}

// ---------------------------------------------------------------- Part C

/** Same index, analyzer and queries for every model; only the scoring function changes. */ private static void runRetrievalModels(
        IndexReader reader, List<String[]> topics, Path resultsDir) throws IOException { Map<String, Similarity> models = new
    LinkedHashMap<>(); models.put("TF-IDF", new ClassicSimilarity()); models.put("BM25", new BM25Similarity(1.2f,
    0.75f)); models.put("LM-Dirichlet", new LMDirichletSimilarity(2000f));
    // Sensitivity variants, reported separately from the default-parameter comparison.
    for (float mu : new float[] {50f, 100f, 200f, 500f, 1000f}) { models.put("LMD-mu" + (int) mu, new
        LMDirichletSimilarity(mu));
    } for (float lambda : new float[] {0.1f, 0.3f, 0.7f}) { models.put("LMJM-l" + lambda, new
    LMJelinekMercerSimilarity(lambda));
    }
    for (float b : new float[] {0.25f, 0.5f, 1.0f}) { models.put("BM25-b" + b, new BM25Similarity(1.2f, b));
    }

    for (var model : models.entrySet()) {
        IndexSearcher searcher = new IndexSearcher(reader); searcher.setSimilarity(model.getValue());
        Path file = resultsDir.resolve("partc_" + model.getKey() + ".run"); try (PrintWriter out = new
        PrintWriter(file.toFile())) {
            for (String[] topic : topics) {
                Query query = orQuery(analyze(topic[2])); writeRun(searcher, query, Integer.parseInt(topic[0]), model.getKey(), 1000,
                out);
            }
        }
    }
}
}
```

ir_metrics.py

```
"""IR evaluation metrics shared by Part B and Part C.

A ranked list is a list of document IDs, best first. 'rel' is the set of relevant IDs.
""" import
math


def precision(ranked, rel): return sum(d in rel for d in ranked) / len(ranked) if ranked else 0.0


def recall(ranked, rel): return sum(d in rel for d in ranked) / len(rel) if rel else 0.0


def f1(ranked, rel):
    p, r = precision(ranked, rel), recall(ranked, rel) return 2 * p * r / (p + r) if p + r else
    0.0


def precision_at_k(ranked, rel, k):
    """The denominator stays k even if fewer than k documents were returned.""" return sum(d in rel for d in ranked[:k]) /
    k


def reciprocal_rank(ranked, rel):
    for rank, doc in enumerate(ranked, 1):
        if doc in rel:
            return 1 / rank return 0.0


def average_precision(ranked, rel):
    """Relevant documents that were not retrieved contribute 0 (division by |rel|)."""" hits, total = 0, 0.0 for rank, doc in
    enumerate(ranked, 1):
        if doc in rel:
            hits += 1 total += hits / rank return total /
    len(rel) if rel else 0.0


def dcg(gains):
    return sum(gain / math.log2(rank + 1) for rank, gain in enumerate(gains, 1))


def ndcg_at_k(ranked, grades, k):
    """ 'grades' maps doc -> gain; unjudged documents have gain 0.

    The ideal list is built from all judged documents, not just the retrieved ones.
    """
    actual = dcg([grades.get(d, 0) for d in ranked[:k]]) ideal =
    dcg(sorted(grades.values(), reverse=True)[:k]) return actual / ideal if ideal else 0.0


def mean(values):
    values = list(values) return sum(values) / len(values) if values else 0.0
```

part_c_eval.py

```
"""Part C: evaluate saved runs with the Part B metric functions and test the differences.

Usage: python part_c_eval.py <data_dir> <results_dir>
""" import collections
import json

import sys

import numpy as np from scipy.stats import
wilcoxon

from ir_metrics import (
    average_precision, f1, mean,
    ndcg_at_k, precision,
    precision_at_k, recall,
    reciprocal_rank,
)

MODELS = ["TF-IDF", "BM25", "LM-Dirichlet"] VARIANTS = [
    "LMD-mu50", "LMD-mu100", "LMD-mu200", "LMD-mu500", "LMD-mu1000",
    "LMJM-l0.1", "LMJM-l0.3", "LMJM-l0.7",
    "BM25-b0.25", "BM25-b0.5", "BM25-b1.0",
]
PAIRS = [("BM25", "TF-IDF"), ("BM25", "LM-Dirichlet"), ("TF-IDF", "LM-Dirichlet")]
TESTED_METRICS = ["AP", "nDCG10", "RR", "P10"]
PERMUTATIONS = 100_000 SEED =
618


def load_qrels(path):
```

```
    """Returns relevant sets and gain dictionaries. This qrel copy is binary, so gain = 1."""" relevant, gains = collections.defaultdict(set),
    collections.defaultdict(dict) for line in open(path): query, _, doc, label = line.split() if int(label) > 0: relevant[query].add(doc)
    gains[query][doc] = 1
    return relevant, gains


def load_run(path):
    """Reads a TREC-format run file into {query: [doc, doc, ...]} in rank order."""" run = collections.defaultdict(list) for line
    in open(path): parts = line.split() run[parts[0]].append(parts[2])
    return run


def evaluate(run, relevant, gains): results = {} for query,
    rel in relevant.items(): ranked = run.get(query, [])
    results[query] = {
                "P": precision(ranked, rel),
                "R": recall(ranked, rel),
                "F1": f1(ranked, rel),
                "P10": precision_at_k(ranked, rel, 10),
                "R10": recall(ranked[:10], rel),
                "F1_10": f1(ranked[:10], rel),
                "P3": precision_at_k(ranked, rel, 3),
                "AP": average_precision(ranked, rel),
                "RR": reciprocal_rank(ranked, rel),
                "nDCG5": ndcg_at_k(ranked, gains[query], 5),
                "nDCG10": ndcg_at_k(ranked, gains[query], 10),
        } return
    results


def average(per_query):
    keys = next(iter(per_query.values())) return {key: mean(scores[key] for scores in per_query.values()) for key in keys}


def paired_tests(per_model, queries):
    """Two-sided sign-flip randomization test and Wilcoxon signed-rank test on each pair."""" rng = np.random.default_rng(SEED) tests
    = [] for a, b in PAIRS:
        for metric in TESTED_METRICS:
            x = np.array([per_model[a][q][metric] for q in queries]) y =
            np.array([per_model[b][q][metric] for q in queries]) diff = x - y
            signs = rng.choice([-1, 1], size=(PERMUTATIONS, len(diff))) p_perm = (np.abs((signs * diff).mean(axis=1)) >=
            abs(diff.mean()) - 1e-12).mean() p_wilcoxon = wilcoxon(x, y, zero_method="wilcox").pvalue if np.any(diff) else 1.0
            tests.append(
                {
                    "a": a,
                    "b": b,
                    "metric": metric,
                    "diff": float(diff.mean()),
                    "wins": int((diff > 0).sum()),
                    "losses": int((diff < 0).sum()),
                    "ties": int((diff == 0).sum()),
                    "p_perm": float(p_perm),
                    "p_wilcoxon": float(p_wilcoxon),
                }
            ) return
    tests


def best_model_counts(per_model, queries):
    counts = collections.Counter() for query in
    queries:
```

```
        scores = {m: per_model[m][query]["AP"] for m in MODELS} top = max(scores.values())
        winners = [m for m in MODELS if abs(scores[m] - top) < 1e-12] counts["tie" if
        len(winners) > 1 else winners[0]] += 1
    return dict(counts)


def largest_spread_examples(per_model, queries, relevant, texts, n=5):
    def spread(query):
        scores = [per_model[m][query]["AP"] for m in MODELS] return -(max(scores) -
        min(scores))

    return [ {
            "q": query,
            "text": texts[query],
            "nrel": len(relevant[query]),
            **{m: per_model[m][query]["AP"] for m in MODELS},
        } for query in sorted(queries, key=spread)[:n]
    ]


def effect_of_raw_query_ids(data, results_dir, relevant, gains):
    """Shows what happens if the non-consecutive query numbers are not renumbered."""
    raw_number = { line.split("\t")[0]: line.split("\t")[1] for line in open(f"{data}/queries.tsv")
    } run = load_run(f"{results_dir}/partc_BM25.run")
    wrong = evaluate({raw_number[q]: docs for q, docs in run.items()}, relevant, gains) return {key: mean(scores[key] for scores in
    wrong.values()) for key in ["AP", "P10", "RR"]}


def write_per_query_csv(path, per_model, queries):
```

```
    keys = list(next(iter(per_model["BM25"].values()))) with open(path, "w") as f:
        f.write("model,query," + ",".join(keys) + "\n") for model in MODELS:
            for query in queries:
                values = ",".join(f"{per_model[model][query][k]:.6f}" for k in keys) f.write(f"{model},{query},{values}\n")


def main(data, results_dir):
    relevant, gains = load_qrels(f"{data}/qrels.txt") queries = sorted(relevant, key=int) texts = {line.split("\t")[0]: line.split("\t")[2].strip() for line in
    open(f"{data}/queries.tsv")
    }

    runs = {m: load_run(f"{results_dir}/partc_{m}.run") for m in MODELS + VARIANTS} per_model = {m: evaluate(runs[m], relevant,
    gains) for m in MODELS + VARIANTS}

    output = {
        "agg": {m: average(per_model[m]) for m in MODELS + VARIANTS},
        "wrong_ids": effect_of_raw_query_ids(data, results_dir, relevant, gains),
        "tests": paired_tests(per_model, sorted(relevant)),
        "best_AP": best_model_counts(per_model, queries),
        "zero_P10": {m: sum(per_model[m][q]["P10"] == 0 for q in queries) for m in MODELS},
        "examples": largest_spread_examples(per_model, queries, relevant, texts),
        "overlap10": { f"{a}|{b}": mean(len(set(runs[a][q][:10]) & set(runs[b][q][:10])) / 10 for q in queries) for a, b in PAIRS
        },
        "retrieved_mean": {m: mean(len(runs[m][q]) for q in queries) for m in MODELS},
    } json.dump(output, open(f"{results_dir}/partc_results.json", "w"), indent=1)
    write_per_query_csv(f"{results_dir}/partc_per_query.csv", per_model, queries)

    for model in MODELS:
        scores = output["agg"][model] print(f"{model:13} MAP={scores['AP']:.4f} MRR={scores['RR']:.4f} nDCG@10={scores['nDCG10
    ']:.4f}")


if __name__ == "__main__":
    main(sys.argv[1], sys.argv[2])
```

verify.py

```
"""Cross-check our Part C metrics against the independent ranx library.

Usage: python verify.py <data_dir> <results_dir>
""" import json
import sys

from ranx import Qrels, Run, evaluate

from part_c_eval import MODELS, load_qrels

METRIC_PAIRS = {
    "map": ("AP", "map"),
    "mrr": ("RR", "mrr"),
    "p3": ("P3", "precision@3"),
    "p10": ("P10", "precision@10"),
    "r10": ("R10", "recall@10"),
    "n10": ("nDCG10", "ndcg@10"),
    "n5": ("nDCG5", "ndcg@5"),
    "r": ("R", "recall@1000"),
}


def read_run(path):
```

```
    run = {} for line in open(path):
        query, _, doc, rank, _score, _tag = line.split() run.setdefault(query, {})[doc] = 1000.0 - int(rank)
    return Run(run)


def main(data, results_dir):
    relevant, _ = load_qrels(f"{data}/qrels.txt") qrels = Qrels({q: {d: 1 for d in docs} for q, docs in
    relevant.items()}) own = json.load(open(f"{results_dir}/partc_results.json"))["agg"]

    comparison = {} for model in
    MODELS:
        theirs = evaluate(qrels, read_run(f"{results_dir}/partc_{model}.run"),
                                                [name for _, name in METRIC_PAIRS.values()])
        comparison[model] = { label: (round(own[model][key], 6), round(float(theirs[name]), 6)) for label,
            (key, name) in METRIC_PAIRS.items()
        } match = all(a == b for a, b in comparison[model].values()) print(f"{model:13} all
        metrics match ranx: {match}")
    json.dump(comparison, open(f"{results_dir}/verify_ranx.json", "w"), indent=1)


if __name__ == "__main__":
    main(sys.argv[1], sys.argv[2])
```